

## 1. Introduction

### 1.1 Introduction

Urban areas are continuously expanding, and cities are facing increasing challenges in managing civic infrastructure and public services. Common civic problems such as potholes, garbage overflow, broken streetlights, damaged roads, water leakage, and blocked drainage systems affect the daily lives of citizens and reduce the overall quality of urban living. These issues not only cause inconvenience but also create serious safety risks and environmental concerns. For example, potholes may lead to road accidents, garbage overflow may cause health hazards, and broken streetlights can increase the chances of crime during nighttime.

Municipal authorities are responsible for identifying and resolving these civic issues. However, traditional complaint systems are often inefficient and time-consuming. Citizens usually report problems through phone calls, office visits, or paper-based applications. These manual approaches are slow and lack transparency. Complaints may not reach the appropriate departments on time, and citizens are often unaware of the progress or status of their reported issues. As a result, many civic problems remain unresolved for long periods.

With the rapid growth of smartphones and internet connectivity, digital platforms can significantly improve communication between citizens and municipal authorities. Mobile applications allow users to capture images of civic problems, automatically attach location information using GPS, and submit complaints instantly. These systems can reduce manual effort, improve response time, and increase transparency.

The proposed system “AI-Driven Civic Issue Reporting System with YOLO-Based Auto Detection” aims to develop a smart digital platform where citizens can easily report civic issues using a mobile or web application. The system allows users to upload images of problems, automatically capture location details, and submit complaints. The system uses YOLO-based object detection to automatically identify the type of civic issue from the uploaded image. This reduces manual classification and helps authorities quickly assign complaints to appropriate departments.

Municipal authorities can access an administrative dashboard to monitor complaints, categorize issues, assign tasks to field workers, and track resolution progress. Citizens can also receive notifications regarding complaint submission, status updates, and resolution details. This system promotes transparency, citizen participation, and efficient governance. By integrating artificial intelligence and digital reporting, the system improves civic issue management and helps municipal authorities respond more effectively.

---

# AI-Driven Civic Issue Reporting System with YOLO Based Auto Detection

---

## 1.2 Problem Description

Cities frequently experience civic problems such as damaged roads, potholes, garbage overflow, broken streetlights, water leakage, and drainage issues. These problems affect public safety, environmental cleanliness, and the quality of life for citizens. Poorly maintained infrastructure may cause accidents, traffic congestion, and health-related problems. With increasing urbanization and population growth, these civic issues are becoming more frequent and difficult to manage.

Despite encountering these problems regularly, citizens often lack a simple and efficient way to report them to municipal authorities. Existing systems are mostly manual and require users to visit municipal offices or make phone calls. These methods are inconvenient and time-consuming. In many cases, complaints are not recorded properly, and there is no tracking mechanism to monitor their progress. Citizens remain unaware of whether their issues are being addressed, which reduces their motivation to report problems.

Municipal authorities also face challenges in managing complaints effectively. Reports are received from multiple channels such as phone calls, emails, and social media platforms. This scattered information makes it difficult to organize, categorize, and prioritize complaints. Authorities must manually analyze each complaint and assign it to the relevant department, which increases workload and delays response time.

Another limitation of existing systems is the lack of automation in issue detection. When users submit complaints, administrators must manually identify the type of problem based on description and images. This manual classification process is time-consuming and may lead to errors. Without proper categorization, complaints may be assigned incorrectly, resulting in delays in resolution.

Therefore, there is a need for a centralized digital system that allows citizens to easily report civic problems while enabling authorities to manage complaints efficiently. The system should automatically detect issue types using AI, store complaint details in a structured format, and provide real-time tracking. Such a system can improve transparency, reduce response time, and support better decision-making for municipal authorities.

## 1.3 Problem Statement

Design and develop an AI-Driven Civic Issue Reporting System that allows citizens to report civic problems using images, GPS location, and descriptions while enabling municipal authorities to monitor complaints, automatically detect issue types using YOLO-based object detection, assign tasks to departments, and track issue resolution through a centralized dashboard.

---

# AI-Driven Civic Issue Reporting System with YOLO Based Auto Detection

---

## 1.4 Objectives

1. To develop a digital platform that allows citizens to report civic issues using images and location information.
2. To implement YOLO-based automatic detection to identify the type of civic issue from uploaded images.
3. To create a centralized dashboard for municipal authorities to monitor and manage complaints.
4. To enable automatic complaint routing to appropriate departments based on issue type.
5. To provide real-time status updates and notifications to citizens.
6. To generate reports and analytics for improved decision-making.
7. To provide multi-language support that allows citizens to use the system in different regional languages, making the platform accessible and easy to use for a wider range of users.

## 1.5 Scope

The software product to be developed is **AI-Driven Civic Issue Reporting System with YOLO-Based Auto Detection**. The system consists of major components including Citizen Interface, Administrative Dashboard, YOLO-Based Detection Module, Complaint Management System, Notification Module, and Database Management System.

The proposed system provides an integrated digital platform for reporting and managing civic issues. Citizens can create profiles, capture images of civic problems, and submit complaints along with location details. The system automatically analyzes uploaded images using YOLO-based object detection to identify the type of issue such as pothole, garbage overflow, broken streetlight, or water leakage. Based on the detected category, the complaint is stored and made available to municipal authorities.

Municipal authorities can access the administrative dashboard to view complaints, categorize them, assign tasks to appropriate departments, and update resolution status. The system also provides notifications to citizens regarding complaint submission, processing, and resolution. This improves transparency and communication between citizens and authorities.

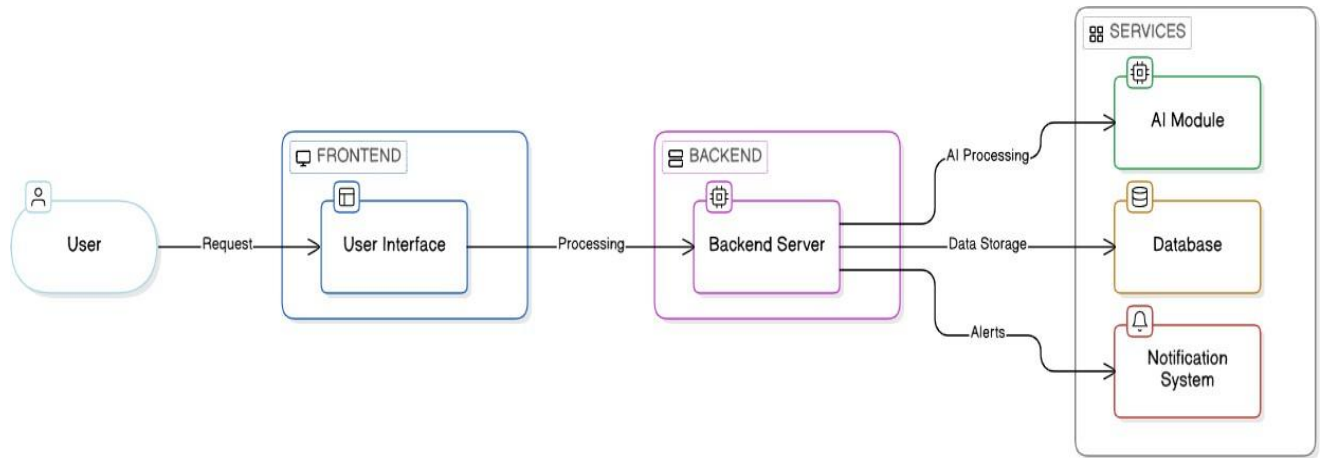
The application of this software lies in smart city governance and civic issue management. It helps improve infrastructure maintenance, reduces manual workload, and increases efficiency in complaint resolution. However, the system does not perform physical repair work and does not replace municipal operations. It acts as a digital support tool for reporting, tracking, and managing civic issues.

---

## 2. Overall description

### 2.1 Product perspective

#### 2.1.1 Block diagram-



**Fig1 : Block Diagram**

The diagram represents the architecture of an **AI-Driven Civic Issue Reporting System**, where users submit complaints through a frontend interface with images, location, and descriptions. The backend server processes these requests, stores data in the database, and coordinates with an AI module that uses YOLO-based detection to automatically identify and classify civic issues. Additionally, a notification system sends updates to users about complaint status, ensuring transparency and efficient communication between citizens and municipal authorities.

#### 2.1.2. Hardware Requirements-

- Processor: Intel Core i5 or higher
  - RAM: Minimum 8 GB
  - Storage: 256 GB SSD or higher
  - Operating System: Windows 10/Linux/macOS
  - Smartphone: Required for capturing civic issue images
-

## 2.1.3. Software Requirements-

- Frontend: React, Tailwind CSS
- Backend: Node.js, Express.js
- AI Development: Python (YOLO)
- Database: MongoDB
- Code Editor: Visual Studio Code
- Design Tool: Draw.io (for diagrams)

## 2.2 Product functions

The AI-Driven Civic Issue Reporting System provides integrated functionalities for citizens and municipal authorities as described below:

1. **User Registration and Login:** The system allows citizens and administrators to register and securely log in to access system features. Users provide personal information and credentials during registration.
  2. **Citizen Complaint Reporting:** The system enables citizens to capture images of civic issues and upload them along with description and location details. The complaint is submitted to the system for processing.
  3. **Automatic Issue Detection:** The system analyzes uploaded images using YOLO-based object detection to identify the type of civic issue automatically.
  4. **Location Tagging:** The system captures GPS location of the reported issue and stores it along with complaint details.
  5. **Complaint Management :** The system stores complaints in the database and displays them on the admin dashboard for monitoring and processing.
  6. **Administrative Dashboard:** The system allows administrators to view complaints, filter them, assign tasks to departments, and update status.
  7. **Notification System:** The system sends notifications to users regarding complaint submission, progress updates, and resolution.
  8. **Complaint Tracking:** Citizens can track the status of their complaints including pending, in progress, and resolved.
-

## 3. Specific requirements

This section describes the detailed requirements of the AI-Driven Civic Issue Reporting System to provide a clear understanding for system designers and developers to design and implement the system effectively. It also helps testers verify that the system meets all specified requirements.

The requirements specified in this section define how the system will respond to various inputs provided by users such as citizens and administrators, and how it will generate appropriate outputs. These include user interactions such as registration, login, complaint submission, image upload, and status tracking.

The system processes these inputs using backend logic, database operations, and AI-based YOLO detection module. Based on the uploaded images, the AI module detects the issue type and stores the result in the database. The system then generates outputs such as complaint confirmation, detected issue category, dashboard display, and notifications.

All functional and non-functional requirements are described to ensure that every operation performed by the system is clearly defined, testable, and aligned with the overall objectives of the project. These requirements help developers understand system behavior and allow stakeholders to verify that the system performs as expected.

### 3.1 External interfaces

The AI-Driven Civic Issue Reporting System interacts with users and external components through various input and output interfaces.

#### Inputs to the System:

- User registration details (name, email, password, role)
  - Login credentials
  - Complaint image uploaded by user
  - GPS location data
  - Complaint description text
  - Admin status update input
  - Complaint assignment details
-

# AI-Driven Civic Issue Reporting System with YOLO Based Auto Detection

---

## Outputs from the System:

- User authentication status (success/failure)
- Complaint submission confirmation
- Detected issue type using YOLO
- Display of complaint details on dashboard
- Status update notifications
- Resolution confirmations
- Reports and analytics

The system ensures proper interaction between frontend interface, backend server, database, and AI detection module. Input data provided by users is processed by the backend and AI module, and outputs are generated in the form of notifications, dashboard updates, and complaint tracking information.

## 3.2 Functional Requirements

The functional requirements of the AI-Driven Civic Issue Reporting System are defined as follows:

1. The system shall allow users to register and log in securely.
  2. The system shall allow citizens to upload images of civic issues.
  3. The system shall automatically capture GPS location of the issue.
  4. The system shall allow users to add description of the issue.
  5. The system shall analyze uploaded image using YOLO detection.
  6. The system shall automatically detect the issue category.
  7. The system shall store complaint details in the database.
  8. The system shall allow administrators to view complaints.
  9. The system shall allow administrators to filter complaints by category.
  10. The system shall allow administrators to assign complaints to departments.
  11. The system shall allow administrators to update complaint status.
  12. The system shall send notifications to users regarding complaint status.
  13. The system shall allow users to track complaint progress.
  14. The system shall display complaint history.
  15. The system shall generate reports for administrators.
-

# AI-Driven Civic Issue Reporting System with YOLO Based Auto Detection

## 3.3 Performance requirements

The AI-Driven Civic Issue Reporting System shall meet both static and dynamic performance requirements to ensure efficient and reliable operation.

### Static Requirements:

- The system shall support multiple users accessing the platform simultaneously.
- The system shall support both mobile and desktop devices.
- The system shall store and manage large numbers of complaint records.
- The system shall support storage of images uploaded by users.
- The system shall handle location data and user information efficiently.
- The system shall maintain proper response time under normal working conditions.

### Dynamic Requirements:

- The system shall respond to user requests within 2–3 seconds under normal conditions.
- The system shall process complaint submissions without delay.
- The YOLO detection module shall analyze uploaded images within a reasonable time.
- The system shall handle multiple complaint uploads simultaneously.
- The system shall allow administrators to update status in real-time.
- The system shall generate notifications without significant delay.
- The system shall handle peak workloads without system failure.

## 3.4 Design constraints

The AI-Driven Civic Issue Reporting System is subject to certain design constraints that may affect development and operation.

- The system shall be developed using standard web technologies such as HTML, CSS, JavaScript, Node.js, and Python for AI module.
  - The system shall be compatible with commonly used operating systems such as Windows and mobile platforms.
  - The system shall depend on internet availability for uploading complaints and accessing dashboard.
  - The YOLO detection module shall depend on dataset availability and model accuracy.
  - The system design shall consider limited hardware resources of mobile devices.
  - The system shall follow secure authentication and data protection mechanisms.
  - The system shall store image data efficiently to reduce storage overhead.
  - The system shall comply with academic project guidelines.
-

## 3.5 Logical database requirements

The AI-Driven Civic Issue Reporting System requires a structured database to store and manage information related to users, complaints, detection results, and status updates.

a) **Types of Information:** The database shall store user information such as citizen and admin details, complaint information including description and uploaded images, GPS location data, detected issue categories generated by the YOLO module, complaint status details, and administrative actions. The system shall also store notification data and complaint history for tracking and reporting purposes.

b) **Frequency of Use:** User data shall be accessed frequently for operations such as registration and login. Complaint data shall be accessed regularly for viewing, filtering, and updating by administrators. Detection results shall be retrieved whenever complaints are processed or displayed. Status information shall be updated frequently during complaint resolution, while notification data may be accessed periodically.

c) **Data Entities and Relationships:** The main entities include User, Admin, Complaint, Location, Issue Type, and Status. A user can submit multiple complaints, and each complaint is associated with one location and one detected issue type. Administrators can manage multiple complaints and update their status. Each complaint progresses through different stages such as pending, in progress, and resolved.

d) **Integrity Constraints:** The system shall maintain data integrity using primary keys, foreign keys, and validation rules to ensure accurate and consistent data storage. The system shall validate user input before storing it in the database. Each complaint shall be uniquely identified, and relationships between users, complaints, and status updates shall be properly maintained.

e) **Data Retention Requirements:** The system shall store user and complaint data for long-term use to support tracking and analysis. Temporary data such as session details may be deleted after logout. Old complaint records may be archived periodically, and resolved complaints shall remain available for future reference and reporting.

---

## 3.6 Software system attributes

The AI-Driven Civic Issue Reporting System shall satisfy the following software quality attributes to ensure efficient, secure, and reliable operation.

### 3.6.1 Reliability

The system shall provide consistent and accurate results under normal operating conditions. It shall minimize system failures through proper error handling, input validation, and stable backend processing. The YOLO detection module shall generate reliable issue classification based on uploaded images. The system should perform its functions accurately over long periods without breakdown. Proper database management and exception handling mechanisms shall be implemented to avoid data loss and ensure smooth system operation.

### 3.6.2 Availability

The system shall be available for users at all times with minimal downtime. Citizens should be able to submit complaints whenever issues occur. The administrative dashboard shall also remain accessible for monitoring and updating complaints. The system shall include backup and recovery mechanisms to ensure continuous operation. In case of server failure, the system should restart and resume normal functioning with minimal interruption.

### 3.6.3 Security

The system shall ensure secure access through authentication and authorization mechanisms. Users shall be required to log in before submitting complaints. The system shall protect user data and complaint information from unauthorized access. Passwords shall be stored securely, and access control shall be implemented for admin features. The system shall prevent unauthorized modifications and ensure safe storage of uploaded images and location data.

### 3.6.4 Maintainability

The system shall be designed in a modular and structured manner to allow easy updates and improvements. Developers should be able to modify system features such as detection module, dashboard interface, and notification system without affecting other components. Proper documentation and coding standards shall be followed. The modular architecture shall help in debugging, testing, and future enhancements.

### 3.6.5 Portability

The system shall be deployable on different platforms such as web browsers and mobile devices. Users should be able to access the system from smartphones, tablets, and desktop computers. The application shall be compatible with commonly used operating systems. The system shall not require significant changes for deployment on different environments.

---

4. Software Design Document

4.1. Structural Design

a. Class Diagrams

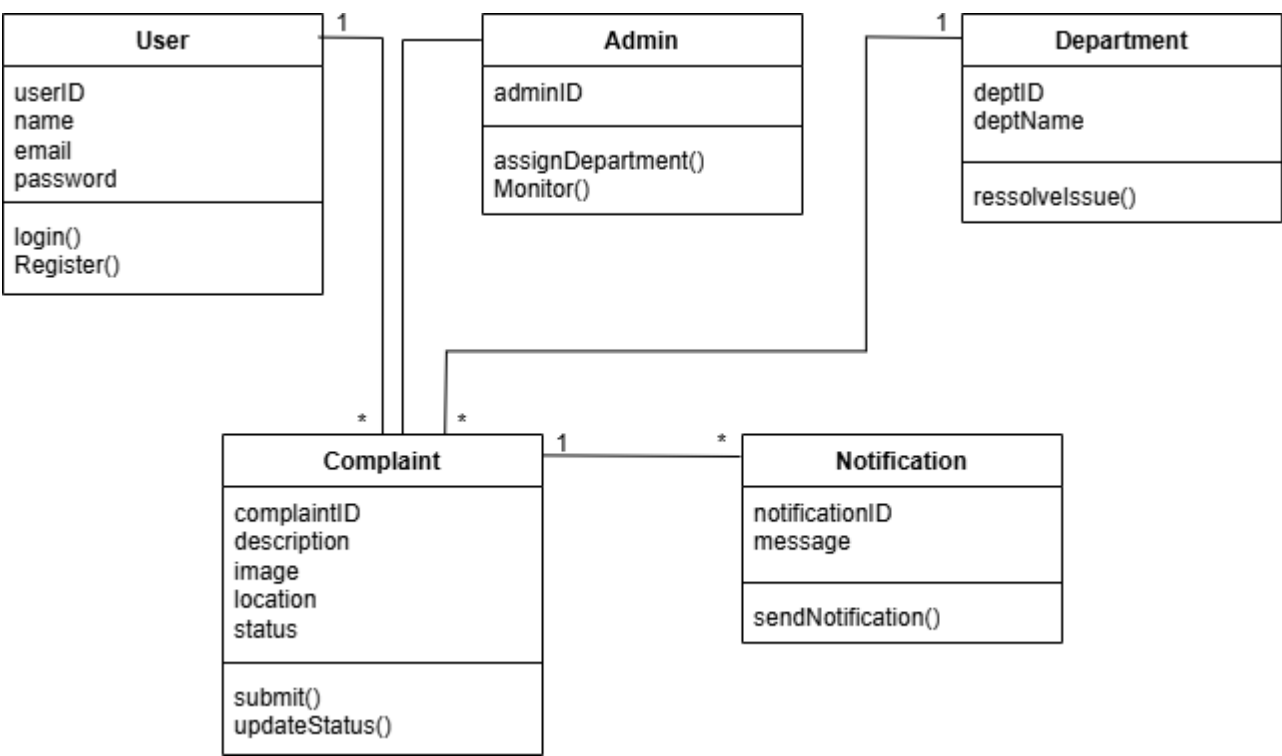


Fig 2: Class Diagram

The diagram represents the class structure of the **AI-Driven Civic Issue Reporting System**, showing how different components interact to manage civic complaints. The User class enables citizens to register, log in, and submit complaints, while the Complaint class stores details such as description, image, location, and status, and allows status updates. The Admin monitors complaints and assigns them to the appropriate Department, which is responsible for resolving issues. The Notification class handles sending updates to users about their complaint status. The relationships indicate that a user can submit multiple complaints, each complaint can generate multiple notifications, and is processed by the admin and relevant department, ensuring efficient complaint handling and communication.

b. Object Diagrams

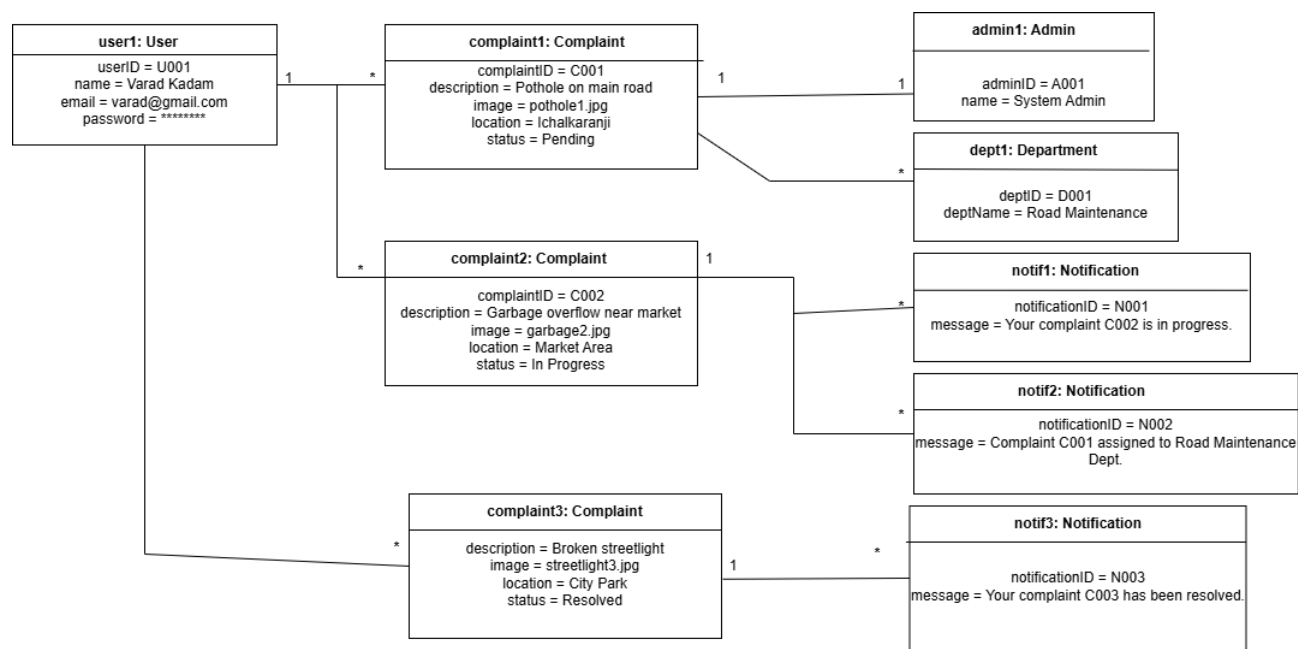


Fig 3: Object Diagrams

The object diagram represents a real-time instance of the **AI-Driven Civic Issue Reporting System**, showing how different objects interact during complaint processing. A specific user (user1) submits a complaint (complaint1) with details like description, image, location, and status. The admin1 object monitors the complaint and assigns it to a relevant department (dept1), which is responsible for resolving the issue. As the complaint progresses, notification objects (notification1, notification2) are generated to inform the user about updates such as assignment and status changes. This diagram provides a snapshot of how system components collaborate dynamically to handle and resolve a civic issue efficiently.

4.2. Data Design  
a. Data-flow Diagram

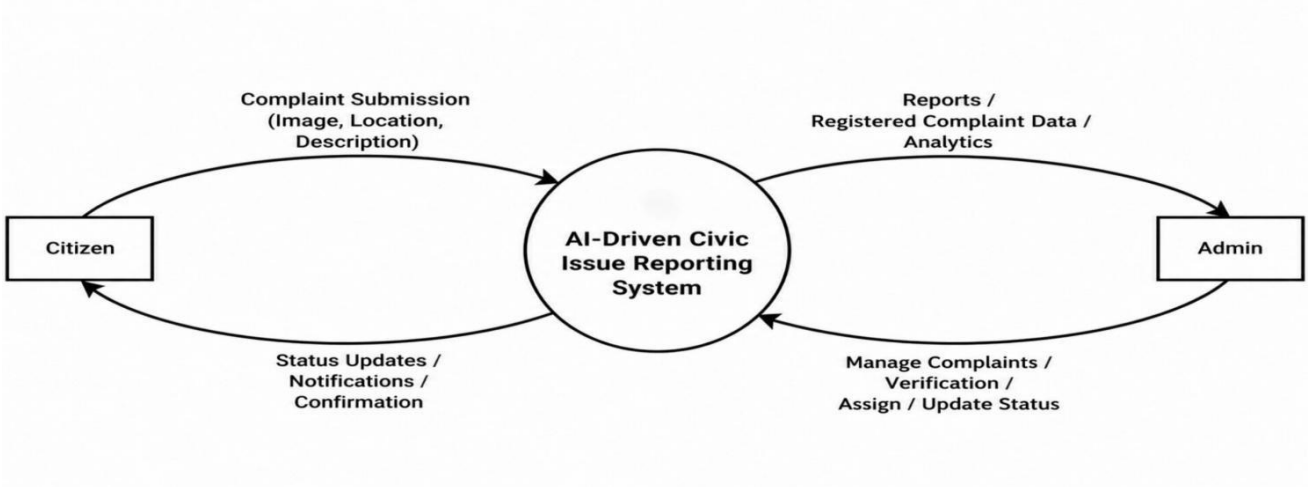


Fig 4:Data-flow Diagram(DFD-0)

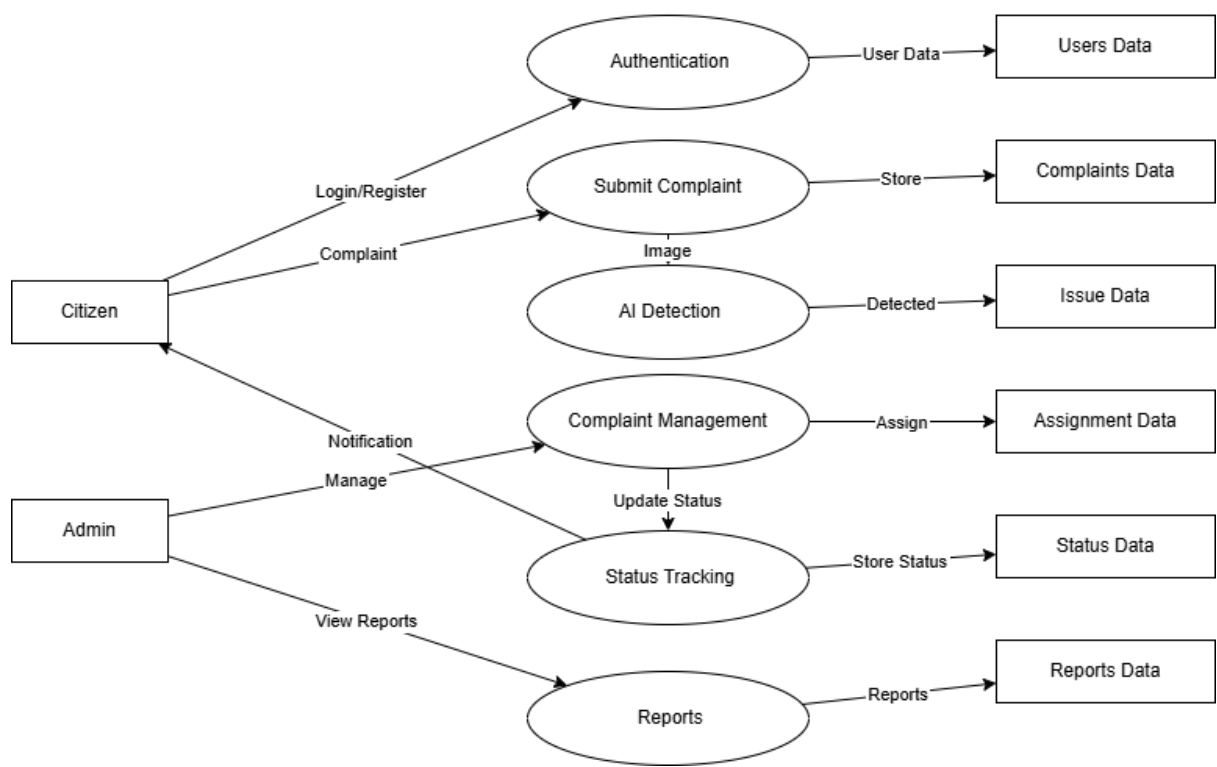


Fig 5:Data-flow Diagram(DFD-1)

The diagrams show the data flow of the AI-Driven Civic Issue Reporting System. Citizens submit complaints, which are verified through user authentication and stored in the complaint database. The AI module automatically classifies issues, and the admin assigns them to the appropriate department. The department resolves the issue and updates the status, which is reflected in the system. Citizens receive real-time notifications about complaint progress. The second diagram illustrates interactions among citizen, admin, department, and AI system, with the AI acting as the central unit for classification, assignment, monitoring, and status tracking.

4.3. Behavioural Design

a. Sequence Diagram

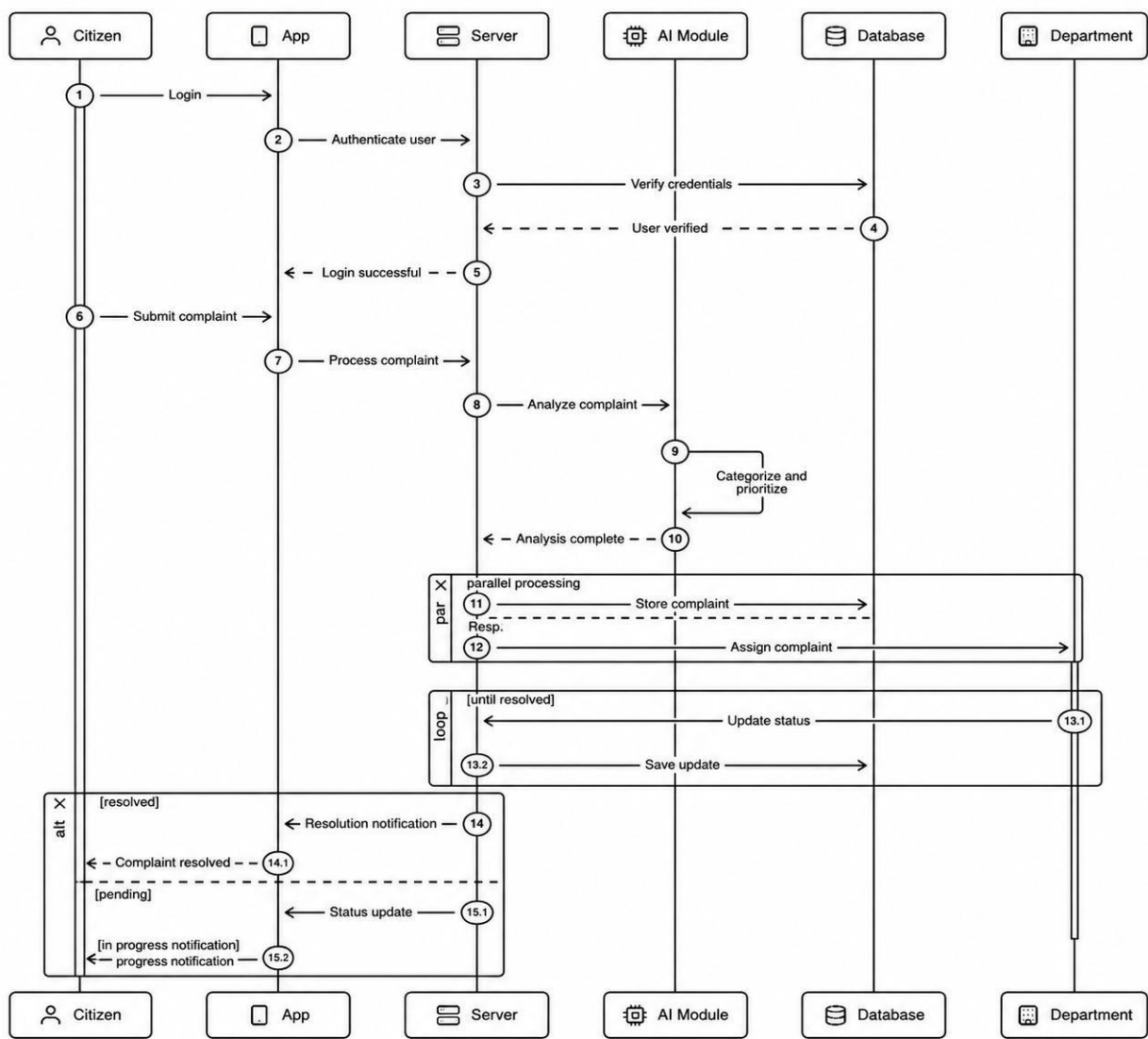
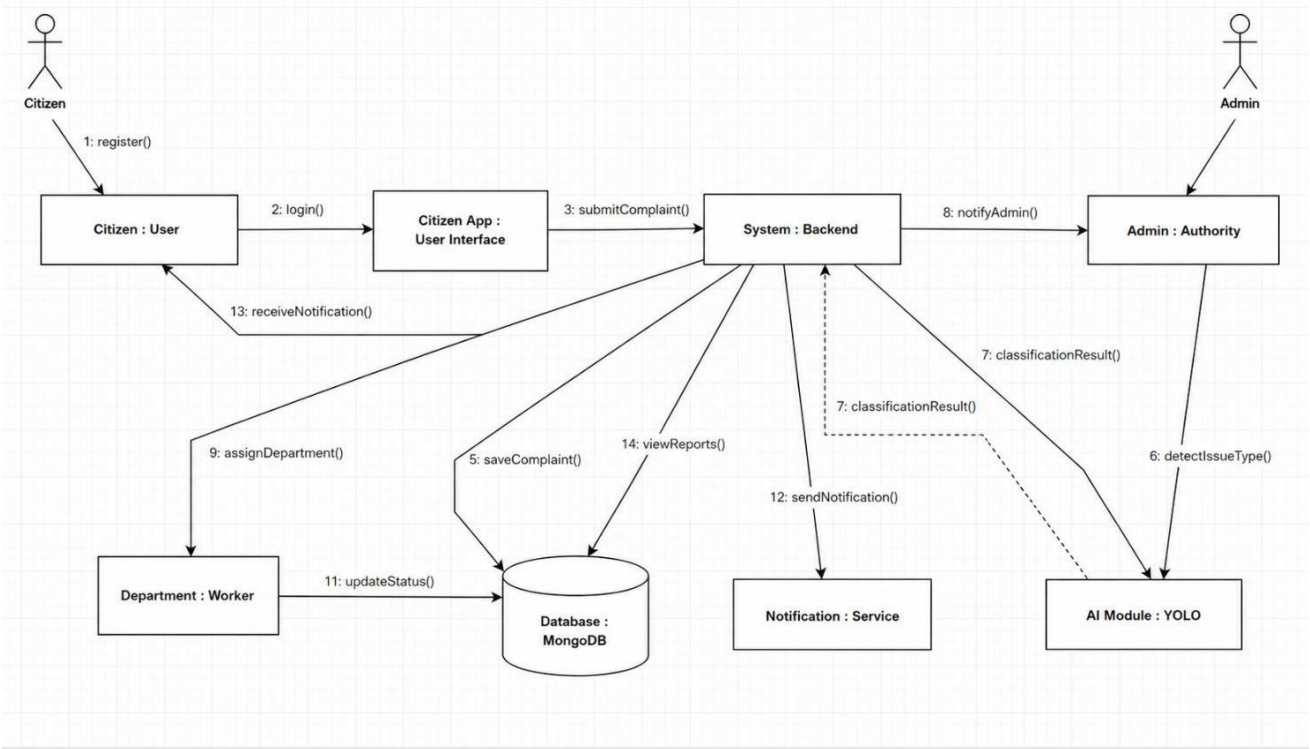


Fig 6: Sequence Diagram

# AI-Driven Civic Issue Reporting System with YOLO Based Auto Detection

The diagram represents the sequence of interactions in the **AI-Driven Civic Issue Reporting System**, showing how different components communicate over time. The process begins when the *Citizen* logs in through the app, and the Server verifies credentials using the database. Once authenticated, the user submits a complaint, which is processed by the server and analyzed by the AI module to categorize and prioritize the issue. After analysis, the complaint is stored in the database and assigned to the appropriate Department. The department works on resolving the issue and continuously updates the status, which is saved in the database. The system then sends notifications to the user regarding progress, such as pending, in-progress, or resolved status. This sequence ensures smooth data flow, automated processing, and real-time communication between all system components.

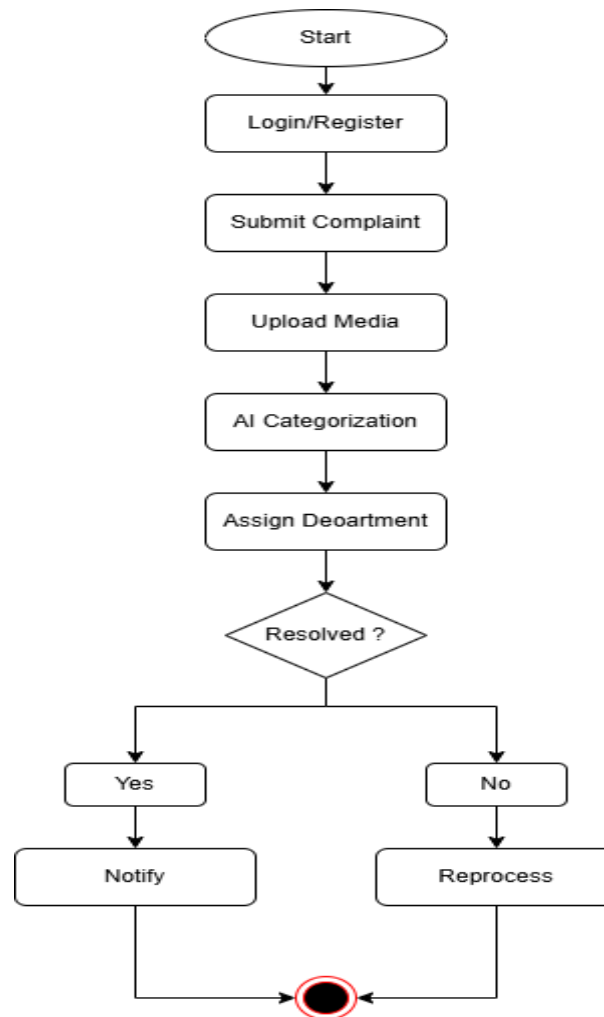
## b. Collaboration diagram



**Fig 7: Collaboration diagram**

The diagram shows the Collaboration Diagram of the AI-Driven Civic Issue Reporting System. The citizen logs in through the app, which communicates with the server for authentication. After login, the complaint is submitted and sent to the server, where the AI module analyzes and classifies it. The complaint is then stored in the database and assigned to the appropriate department for resolution. The department processes the issue, while confirmations and notifications are exchanged between components, ensuring efficient complaint handling.

## c. Activity Diagram



**Fig 8: Activity diagram**

The diagram represents the **activity flow** of the AI-Driven Civic Issue Reporting System, showing the step-by-step process of complaint handling. The process starts with the user logging in or registering, followed by submitting a complaint and uploading relevant media such as images. The system then uses AI to categorize the issue and assigns it to the appropriate department. After assignment, the system checks whether the issue is resolved or not. If resolved, the user is notified about the completion of the complaint. If not, the complaint goes through a reprocessing stage for further action. This flow ensures a structured and continuous process for efficient issue reporting, analysis, and resolution.

---

d. State-chart diagram

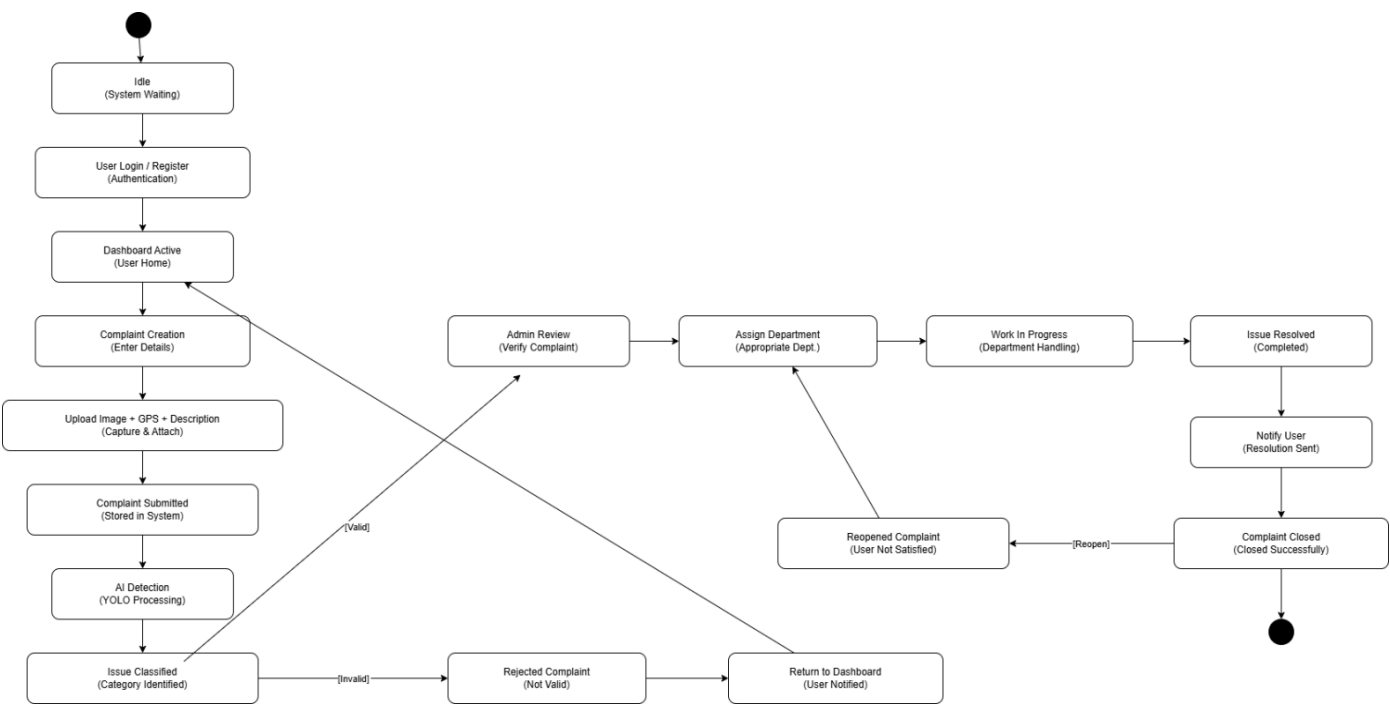


Fig 9: State-chart diagram

The diagram shows the state transition of a complaint in the AI-Driven Civic Issue Reporting System. A complaint starts in the Submitted state, moves to Under Review for validation, and then is Assigned to the appropriate department. The department handles the issue in the In Progress state, after which it becomes Resolved. Finally, the complaint reaches the Closed state, completing the lifecycle and ensuring proper tracking from submission to closure.

4.4. Implementation Design

a. Component Diagram

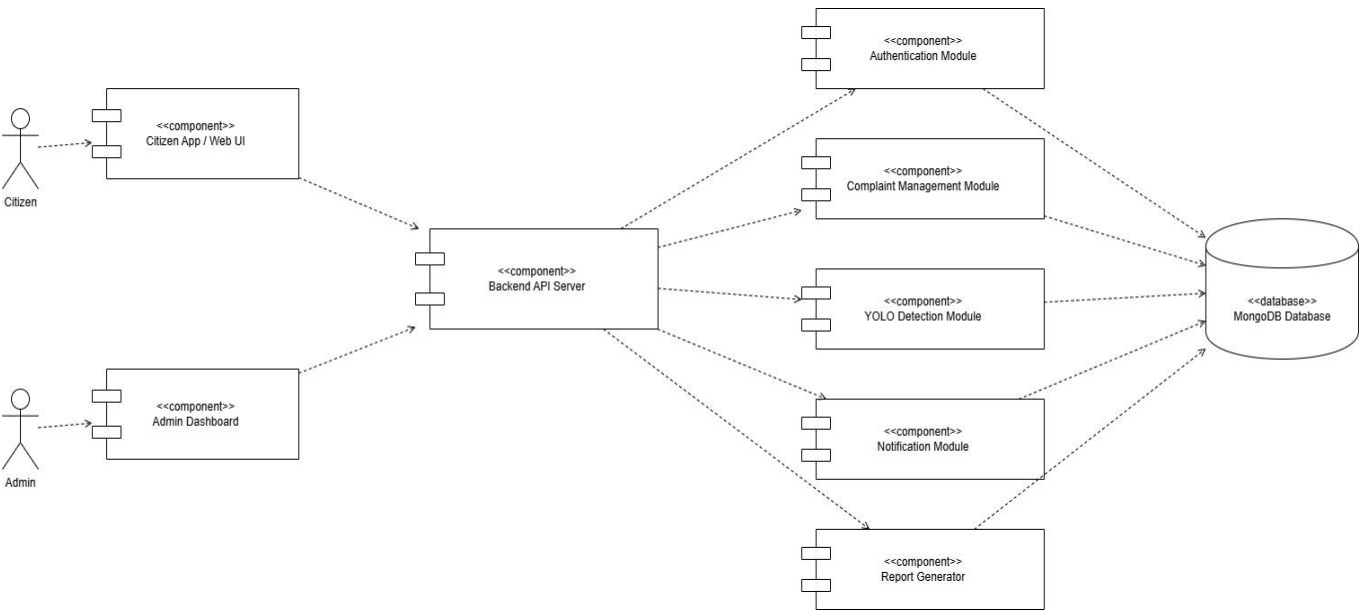
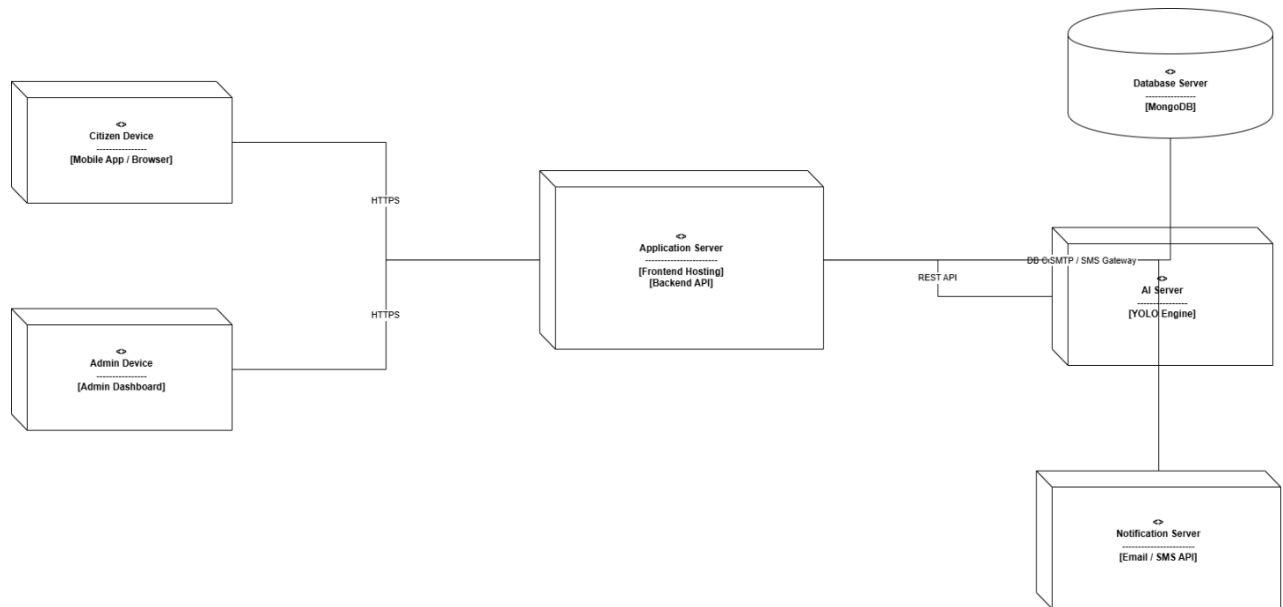


Fig 10: Component diagram

The diagram represents a simplified architecture of the AI-Driven Civic Issue Reporting System, showing the interaction between major system components. The Frontend acts as the user interface through which citizens submit complaints and interact with the system. These requests are sent to the Backend, which serves as the central processing unit, handling business logic and coordinating communication between components. The backend stores and retrieves data from the Database, processes complaints using the AI Module for automatic classification and analysis, and triggers the Notification system to send updates to users. This architecture ensures smooth data flow, efficient processing, and real-time communication across the system.

## 4.5. Environment Design

### a. Deployment Diagram



**Fig 11: Deployment diagram**

The diagram represents the **deployment architecture** of the AI-Driven Civic Issue Reporting System, showing how different hardware and server components are connected. The process begins from the User Device, where users interact with the system through a web or mobile interface. Requests are sent to the Web Server, which handles client communication and forwards them to the Application Server. The application server processes the business logic and communicates with both the Database Server to store and retrieve complaint data, and the AI Server to perform analysis and classification of reported issues. This architecture ensures efficient distribution of tasks across servers, enabling scalable processing, secure data management, and intelligent decision-making within the system.

## 5. Algorithms for Functional Requirements

The algorithms corresponding to the functional requirements of the AI-Driven Civic Issue Reporting System system are described as follows:

### 1. User Registration and Login Algorithm

**Input:** User details (name, email, password, role)

**Process:**

- Accept user input
- Validate input fields
- Store user data in database (for registration)
- Verify credentials with database (for login)

**Output:** Successful registration or login access / error message

### 2. Complaint Submission Algorithm

**Input:** Complaint image, description, location details

**Process:**

- Accept complaint data from user
- Capture GPS location
- Upload image to server
- Store complaint details in database

**Output:** Complaint successfully registered.

### 3. YOLO-Based Issue Detection Algorithm

**Input:** Uploaded complaint image

**Process:**

- Receive image from backend
- Preprocess image
- Apply YOLO model

# AI-Driven Civic Issue Reporting System with YOLO Based Auto Detection

---

- Detect issue type
- Return detected category

**Output:** Detected civic issue type.

## 4. Complaint Management Algorithm

**Input:** Complaint data

**Process:**

- Retrieve complaint from database
- Display complaint on admin dashboard
- Assign complaint to department
- Update complaint status

**Output:** Updated complaint information

## 5. Notification Algorithm

**Input:** Status update

**Process:**

- Detect status change
- Generate notification message
- Send notification to user

**Output:** User receives status notification.

## 6. Complaint Tracking Algorithm

**Input:** User request

**Process:**

- Retrieve complaint data
- Display current status
- Show complaint history

**Output:** Complaint tracking information

---

## 7. Admin Management Algorithm

**Input:** Admin actions **Process:**

- Manage user data
- Monitor system activities
- Update system content

**Output:** Updated system data and control.

## 6. References

- [1] G. T. Reddy et al., "Civic Complaint Tracker," International Journal of Science, Engineering and Technology.
  - [2] R. Kumar et al., "CivicFix: Smart Complaint Routing for Urban Solutions," International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE).
  - [3] R. Kulkarni, N. Kumaresh, and V. L. M., "Civic Drishti: A Crowdsourced Priority Engine for Transparent and Accountable Civic Issue Governance."
  - [4] D. Walwadkar, J. Patil, M. Hussain, and S. Yadav, "Smart Civic Issue Reporting System," Department of Information Technology, Sinhgad College of Engineering, Pune.
  - [5] Monteiro et al., "A Web and Mobile City Maintenance Reporting Solution," Procedia Technology, Elsevier, 2013.
-